



PowerShell

This is a quick reference cheat sheet to getting started with PowerShell scripting.



Star on GitHub

10.7k



Follow us on X



Buy Me a Coffee

Basic Commands

Helper Commands

Powershell follows Verb-Noun format for their commands.

Some common Verbs:

Get	Used to retrieve information.
Set	Used to configure or change settings.
New	Used to create new instances of objects.
Remove	Used to delete or remove items.
Invoke	Used to execute a specific action or operation.

Start	Used to initiate a process or operation.
Stop	Used to halt or terminate a process or operation.
Enable	Used to activate or enable a feature.
Disable	Used to deactivate or disable a feature.
Test	Used to perform tests or checks.
Update	Used to update or refresh data or configurations.

Lists available modules

```
Get-Module --ListAvailable
```

Lists available cmdlets and functions.

```
Get-Command -Module ActiveDirectory
```

Retrieves help

```
Get-Help <cmd>
Get-Help <cmd> -Examples
Get-Help -Name Get-Process -Parameter Id
```

Lists aliases and their corresponding cmdlet names.

```
Get-Alias | Select-Object Name, Definition
```

Get-Member: Displays the properties and methods of objects.

Select-Object: Selects specific properties from objects or customizes their display.

```
Get-Process | Select-Object Name, CPU
```

Where-Object: Filters objects based on specified conditions.

```
Get-Service | Where-Object { $PSItem.Status -eq 'Running' }  
#OR  
Get-Service | ? { $_.Status -eq 'Running' }
```

Measure-Object: Calculates statistics, like sum, average, and count, for object properties.

```
Get-Process | Measure-Object -Property WorkingSet -Sum
```

ForEach-Object: Performs an operation on each object in a collection. (BEAWARE: Below command will prefix of files/folder in the current dir)

```
Get-ChildItem | ForEach-Object { Rename-Item $_ -NewName "Prefix_$_" }
```

Sort-Object: Sorts objects by specified properties.

```
Get-ChildItem | Sort-Object Length -Descending
```

Format-Table: Formats output as a table with specified columns.

```
Get-Service | Format-Table -AutoSize # ft alias
```

Format-List: Formats output as a list of properties and values.

FileSystem

```
New-Item -path file.txt -type 'file' -value 'contents'  
New-Item -path file.txt -type 'dir'  
Copy-Item <src> -destination <dest>
```

```
Move-Item -path <src> -destination <dest>
Remove-Item <file>
Test-Path <path>
Rename-Item -path <path> -newname <newname>
```

```
# using .NET Base Class Library
```

```
[System.IO.File]::WriteAllText('test.txt', '')
```

```
[System.IO.File]::Delete('test.txt')
```

```
Get-Content -Path "test.txt"
```

```
Get-Process | Out-File -FilePath "processes.txt" # Output to file
```

```
Get-Process | Export-Csv -Path "processes.csv" # Output to csv
```

```
$data = Import-Csv -Path "data.csv" # Import from csv
```

System Management

Windows Management Instrumentation

```
# Retrieve BIOS information
```

```
Get-CimInstance -ClassName Win32_BIOS
```

```
# Retrieve information about locally connected physical disk devices
```

```
Get-CimInstance -ClassName Win32_DiskDrive
```

```
# Retrieve information about install physical memory (RAM)
```

```
Get-CimInstance -ClassName Win32_PhysicalMemory
```

```
# Retrieve information about installed network adapters (physical + virtual)
```

```
Get-CimInstance -ClassName Win32_NetworkAdapter
```

```
# Retrieve information about installed graphics / video card (GPU)
```

```
Get-CimInstance -ClassName Win32_VideoController
```

```
# List all the classNames
```

```
Get-CimClass | Select-Object -ExpandProperty CimClassName
```

```
# Explore the various WMI classes available in the root\cimv2 namespace
```

```
Get-CimClass -Namespace root\cimv2
```

```
# Explore the child WMI namespaces underneath the root\cimv2 namespace
```

```
Get-CimInstance -Namespace root -ClassName __NAMESPACE
```

Network Management

```
# Test network connectivity to a remote host
Test-Connection -ComputerName google.com
```

```
# Retrieve network adapter information
Get-NetAdapter
```

```
# Retrieve IP address information
Get-NetIPAddress
```

```
# Retrieve routing table information
Get-NetRoute
```

```
# Test if a port is open on a remote host
Test-NetConnection google.com -Port 80
```

User & Group Management

```
# Retrieve local user account information
Get-LocalUser
```

```
# Create a new local user account
New-LocalUser -Name NewUser -Password (ConvertTo-SecureString "Password123"
-AsPlainText -Force)
```

```
# Remove a local user account
Remove-LocalUser -Name UserToRemove
```

```
# Retrieve local group information
Get-LocalGroup
```

```
# Add a member to a local group
Add-LocalGroupMember -Group Administrators -Member UserToAdd
```

Security & Permissions

```
# Retrieve access control lists for file/dir
Get-Acl C:\Path\To\File.txt
```

```
"Get-ItemProperty -Path 'HKLM:\Software\MyApp' -Name 'SettingName' -Value 'NewValue'" | Out-Null
```

Registry Management

```
# Retrieve registry key values
Get-ItemProperty -Path
"HKLM:\Software\Microsoft\Windows\CurrentVersion\Uninstall\*" | Select
DisplayName, DisplayVersion

# Set registry key values
Set-ItemProperty -Path "HKLM:\Software\MyApp" -Name "SettingName" -Value
"NewValue"

# Create a new registry key value
New-ItemProperty -Path "HKCU:\Software\MyApp" -Name "NewSetting" -Value
"NewValue" -PropertyType String

# Remove a registry key value
Remove-ItemProperty -Path "HKCU:\Software\MyApp" -Name "SettingToRemove"

# Check if a registry key exists
Test-Path "HKLM:\Software\MyApp"
```

Scripting

Variables

Initializing a variable with/without a specified type:

```
$var = 0
[int] $var = 'Trevor'           # (throws an exception)
[string] $var = 'Trevor'       # (doesn't throw an exception)
$var.GetType()
```



```
# Multiple Assigning
$a,$b,$c = 'a','b','c'

# Create an array
$arrayvar = @('va1','va2')

# Create dict
$dict = @{k1 = 'test'; k2 = 'best'}
```

Variable Commands

```
New-Variable -Name FirstName -Value Trevor
New-Variable FirstName -Value Trevor -Option <ReadOnly/Constant>

Get-Variable
Get-Variable | ? { $PSItem.Options -contains 'constant' }
Get-Variable | ? { $PSItem.Options -contains 'readonly' }

Remove-Variable -Name firstname
# Removes ReadOnly var
Remove-Variable -Name firstname -Force
```

Variable types int32, int64, string, bool

Operators

```
# operators
# (a <op> b)

= , += / -= , ++ / --
-eq / -ne , -lt / -gt , -le / -ge

$FirstName = 'Trevor'
$FirstName -like 'T*'
$true; $false #bool true/false

# ternary operator
$FoodToEat = $BaconIsYummy ? 'bacon' : 'beets'

# -notin or -in
'Celery' -in @('Bacon', 'Sausage', 'Steak')
```

```
# output: True
5 -is [int32]

# regex match, array can be use
'Trevor' -match '^T\w*'

# Find multiple matches.
$regex = [regex]'\w*'
$regex.Matches('this is test').Value
```

Structure

I/O operation

```
"This displays a string"

Write-Host "color" -ForegroundColor Red

$page = Read-host "Enter age"

$pwd = Read-host "password" -asSecureString

Clear-Host
```

Flow Controls

```
IF(<#Condition#>){
<#Commands#>}ELSEIF(){ }ELSE{ }

Switch($var){
    "val1"{<#Commands#>; break}
    "val2"{<#Commands#>; break}
}

For($ct=0;$ct -le 3;$ct++){ }

ForEach($var in $arr){ }

while($var -ne 0){ }
```

```
Do{}While()
```

Function / Modules

Example 1

```
function funcname{  
  
    [CmdletBinding()]  
    param(  
        [Parameter(Mandatory)]  
        [String]$user  
    )  
    Write-Host "welcome " $user  
    return "value"  
}  
$var = funcname -user pcb
```

Example 2

```
function Get-EvenNumbers {  
    [CmdletBinding()]  
    param (  
        [Parameter(ValueFromPipeline = $true)]  
        [int] $Number  
    )  
    begin {<#command#>}  
    process {  
        if ($Number % 2 -eq 0) {  
            Write-Output $Number  
        }  
    }  
    end {<#command#>}  
}  
1..10 | Get-EvenNumbers
```

Modules

```
# PowerShell looks module in the path  
$env:PSModulePath
```

```
# lists all modules installed on system
Get-Module -ListAvailable
# modules imported into current session
Get-Module

Import-Module <moduleName>
Remove-Module <moduleName>

Find-Module -Tag cloud
Find-Module -Name ps*

# Create an in-memory PowerShell module
New-Module -Name trevor -ScriptBlock {
    function Add($a,$b) { $a + $b } }
```

Tips

- In most of the languages, escape character is backslash \ whereas in PowerShell it is backtick `

Also see

[Microsoft PowerShell](#) (learn.microsoft.com)

Top Cheatsheet



Python Cheatsheet
Popular Quick Reference

Vim Cheatsheet



Popular Quick Reference



JavaScript Cheatsheet

Popular Quick Reference

Recent Cheatsheet



SBTI Cheatsheet

Latest Quick Reference



Cheatsheet

Latest Quick Reference



Socket io Cheatsheet

Latest Quick Reference

Developer Tools



TableConvert

Convert tables between formats



Referral.is

Best referral codes & bonuses



Fiona AI

Chrome AI assistant



Share quick reference and cheat sheet for developers.

About

Notes

Support

